Fleet-Orthogonal Adaptation: Poison-Resilient On-Device Intrusion Detection for Drifting IoT

Rishit Kamboj and Farhan Naik
Department of Computer Science and Engineering, Shiv Nadar University
CSD457 Internet of Things, Group G-04

***Abstract*—An intrusion detector that runs on an IoT device must keep learning, because normal traffic drifts. But a detector that labels its own traffic can be walked toward an attack by a slow poisoner, and a defence that stops learning when traffic looks suspicious can be frozen by an attacker who trips it and leaves. From a single device's stream, honest drift and slow poisoning cannot be told apart. We observe that IoT devices rarely live alone: most honest drift is shared by a device's neighbours, while a poisoning campaign reaches only the devices its exploit reaches. Fleet-Orthogonal Adaptation (FOA) gives each device a robust consensus of vulnerability-diverse peers, exchanged as 92-byte authenticated ESP-NOW frames: the part of a device's update its peers back is applied in full, and the device-specific rest passes through a budget that never closes. We prove that an attacker on one device gains at most $B_0 + Bt$ alarm radii of movement, that key-holding Byzantine peers add a bounded bias per cycle, that FOA has no freeze state, and that device-only drift and poisoning are indistinguishable for any rule. On all nine N-BaIoT devices with real Mirai and BASHLITE traffic, under strong shared drift, FOA keeps fleet false alarms at 9.0% (single-device adaptation 24.6%, gated defences 45% to 59%) while cutting attack-attributable compromise from 39.2% to 10.8%; it is never compromised by the classic boundary poisoner, resists a monoculture botnet that defeats same-model consensus, and is unaffected by a freeze attack that locks an alarm-rate switch-off at 100% false alarms. On 40 weeks of real CESNET traffic it is never compromised. A C implementation matches the reference on every recorded cycle and adds about 22 KB of flash to an ESP32 build.**

***Index Terms*—IoT security, intrusion detection, data poisoning, concept drift, on-device learning, robust aggregation, ESP32**

I. Introduction

Intrusion detection is moving onto IoT devices themselves: a detector that runs on the device sees its traffic first, needs no upstream collector, and keeps working when the network is cut off. Such a detector has to keep learning, because normal traffic does not stand still. Gateways change, tunnels add bytes, cloud services change how often they poll, firmware updates change what a device sends. A detector frozen at deployment drowns in false alarms within weeks.

Learning on the device has a well-known weak point. The detector labels its own traffic: whatever it does not alarm on becomes the new normal. An attacker who controls part of that traffic can move the boundary a little at a time until an attack looks normal, the boiling-frog attack [1], [2]. The obvious defence, stop learning when traffic looks suspicious, creates a second weakness that we call the *freeze attack*: an attacker trips the stop condition with loud traffic and leaves, honest drift continues, and the frozen detector raises false alarms for good. Recent theory shows the dilemma runs deeper: from a single label-free stream, honest drift and slow contamination can be made indistinguishable [22], [23].

Our observation is that IoT devices rarely live alone. A doorbell, a thermostat and a camera on the same network experience the same gateway, the same tunnel, the same polling changes: most honest drift is *common-mode*. A poisoning campaign, in contrast, reaches only the devices the attacker can exploit, and devices from different vendors do not share exploits. The fleet is a second witness that a single device does not have.

Fleet-Orthogonal Adaptation (FOA) turns that observation into an update rule (Figure 1). Every device keeps its own detector and, each cycle, broadcasts its proposed update as a 92-byte authenticated ESP-NOW frame. On each device the part of its proposal that a robust consensus of vulnerability-diverse peers backs is applied in full; the device-specific remainder, where a single-device campaign lives, passes through a cumulative budget that grows forever. Nothing is discarded and learning never stops, so there is no state for a freeze attack to reach.

Contributions.

- **FOA**, a never-freezing, label-free update rule that splits each on-device update into a peer-backed common part and a budgeted residual, with agreement measured robustly so that an attacker holding less than half a device's traffic can neither cancel nor fake it (Section III).
- **Guarantees**: the attacker's extra movement is bounded by $B_0 + Bt$ alarm radii, giving a minimum time to compromise; $f < K/2$ key-holding Byzantine peers add only a bounded bias per cycle (and alone never compromised a device in our runs); no input stops learning; and a proof that device-only drift and poisoning cannot be separated by any rule, so FOA's single rate B is the honest place to put the trade-off (Section IV).
- **Evaluation on all nine N-BaIoT devices** with real Mirai and BASHLITE traffic, physically modelled drift, eight baselines and held-out seeds. Under strong shared drift FOA keeps fleet false alarms at 9.0% (single-device adaptation 24.6%, gated defences 45% to 59%), and cuts attack-attributable compromise from 39.2% to 10.8% (Section V).
- **The freeze attack** on stop-learning gates: decoys leave FOA's trajectory bit-for-bit unchanged while they lock an alarm-rate switch-off at 100% false alarms.

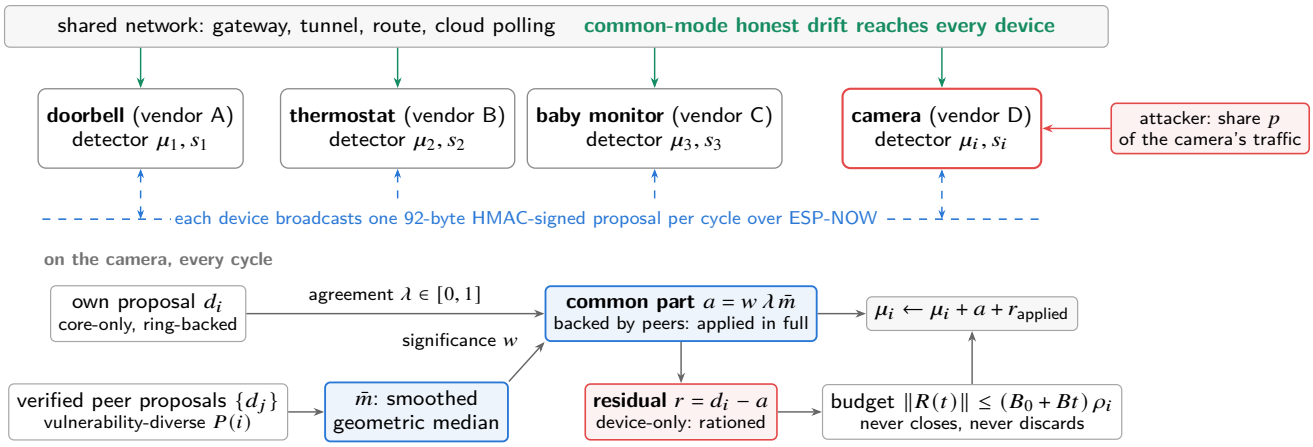


Fig. 1. Fleet-Orthogonal Adaptation. Every device runs its own detector and broadcasts its proposed update each cycle. On each device, the part of its proposal that a robust consensus of vulnerability-diverse peers backs is applied in full; the device-specific remainder, which is where a single-device poisoning campaign lives, passes through a cumulative budget that grows forever, so learning never stops and the attacker's extra movement is bounded.

- **Real drift** on 40 weeks of CESNET traffic [5], and **an ESP32 implementation** that matches the reference on every recorded cycle, rejects all tampered frames and adds about 22 KB of flash.

II. System and Threat Model

A. Devices and detectors

A fleet of IoT devices sits behind a shared network (a home, a campus wing, a factory cell). Each device i runs its own detector on its own traffic. Every packet is summarised by Kitsune's damped incremental statistics [3] (packet sizes, rates and inter-packet gaps at five time scales), sign-log transformed, standardised with fleet-wide constants and projected onto a fleet-shared, whitened basis of dimension $D = 16$. The shared basis is what lets devices compare updates, and sixteen numbers fit easily in one ESP-NOW frame. The detector is the online centroid detector studied by Kloft and Laskov [2]:

$$\text{score}_i(x) = \|x - \mu_i\| / s_i, \qquad \text{alarm if score}_i(x) > \tau_i, \quad (1)$$

with centre μ_i, spread s_i and threshold τ_i set at the 99th percentile of clean calibration scores. We write μ_i^0, s_i^0 for the calibrated values and $\rho_i = \tau_i s_i^0$ for the calibrated alarm radius, the natural unit of how far a detector can be moved.

B. Self-labelled adaptation

Learning runs in cycles. In cycle t, device i observes a batch $X_i(t)$, treats the points it does not alarm on as normal, and proposes an update $(d_i(t), e_i(t))$: a centre shift and a change of log-spread. We use our signature-free rule as the local proposal: it learns only from the dense core of accepted traffic (inside the clean 80th-percentile radius), scales a centre move by how much the outer ring of the cloud moves with it, and allows spread growth only in proportion to outer-ring support. Without a fleet rule, a device applies its proposal in full (*single-device* adaptation). This is the setting in which boiling-frog poisoning works [1], [2]: poison that the detector accepts moves the boundary, which lets the next poison sit further out.

C. Honest drift

Normal traffic changes. We separate two kinds. *Common-mode drift* reaches many devices at once because they share the network: a gateway or tunnel adds bytes to every packet, a route change alters jitter, a cloud service changes its polling rate. *Device-only drift* reaches one device: its own firmware update or a change in how it is used. A detector that cannot follow drift drowns in false alarms; one that follows everything can be poisoned.

D. Threat model

The attacker wants an attack class (for example a Mirai scan) to stop being detected on a victim device, or wants a defence to stop learning so honest drift turns into a flood of false alarms.

- **Traffic share.** The attacker injects traffic that makes up a share p of the victim's learning batch (Kloft and Laskov's attacker share [2]), up to $p = 0.4$. It crafts feature vectors freely but has no code execution on the detector and no keys.
- **Knowledge.** White-box on the algorithm and on the victim's live state (μ, s, the consensus), which is worst case for us.
- **Monoculture botnet.** One exploit can compromise every device that shares a firmware and poison all of them in a coordinated way.
- **Byzantine peers.** Up to f fleet members are fully compromised: the attacker holds their keys and sends any frame it likes.
- **Network attacker.** Without keys it can forge, replay or drop frames.

We call a victim *compromised* when fewer than half of held-out real attack packets of the target class raise an alarm. Out of scope: a compromised gateway that rewrites every device's traffic, and a botnet that controls a majority of every peer group (we show where FOA breaks in that case).

III. Fleet-Orthogonal Adaptation

A. Idea

A single device cannot tell honest drift from slow contamination; with one label-free score stream the two can be made identical [22]. A fleet adds a second, independent stream: the device's peers. Common-mode drift shows up in most peers' proposals, while a campaign that rides one exploit reaches only the peers that share it. FOA therefore splits every proposal into the part the peers back, which is applied in full, and the remainder, which is device-specific and is admitted only through a cumulative budget. Nothing is ever discarded and nothing ever switches learning off, so there is no state for a freeze attack to trip.

B. Peer groups

Each device listens to a peer group $P(i)$ of K devices chosen to be *vulnerability-diverse*: other vendors and firmware lines than its own. Diversity is the point. Same-model peers, as in CIoTA [6], share the very exploit a monoculture botnet uses, so a botnet can manufacture their consensus; diverse peers share the network but not the exploit.

C. The update

Algorithm 1 gives the rule. Each cycle, device i computes its proposal (d_i, e_i), signs it and broadcasts it, and collects its peers' verified proposals $\{(d_j, e_j)\}_{j \in P(i)}$.

1) **Consensus.** m_t is the geometric median of the peers' centre shifts (ten Weiszfeld iterations from the coordinate-wise median), and $\bar{m}_t$ its exponential moving average (weight $\alpha = 0.2$). The geometric median has breakdown point $1/2$: fewer than half the peers, however extreme, move it only boundedly [9], [10].
2) **Significance.** Peers' proposals are noisy even without drift. The weight $w_t \in [0, 1]$ is a ramp on $\|\bar{m}_t\|$ divided by the noise level of $\bar{m}_t$, estimated from the median distance of peer proposals to m_t; it is 0 below two noise levels and 1 above four. It depends on peer frames only.
3) **Agreement.** $\lambda_t \in [0, 1]$ is how much of the smoothed consensus the device's own traffic supports. It is measured robustly: $g_t = \eta \,\mathrm{median}_x \langle x - \mu, \hat{m} \rangle$ over the device's core points, where $\hat{m}$ is the consensus direction, smoothed to $\bar{g}_t$, and $\lambda_t = \mathrm{clip}(\bar{g}_t / \|\bar{m}_t\|, 0, 1)$. A device that does not see the fleet's drift does not take it; an attacker holding less than half the batch can neither cancel nor fake it (an earlier mean-based version could be cancelled, Section V-H); and λ_t is the *only* quantity the victim's own traffic controls.
4) **Common part.** $a_t = w_t \lambda_t \bar{m}_t$, applied in full but kept within a generous common budget $\|C(t)\| \le (B_0^c + B^c t)\rho_i$, where C is the accumulated common movement.
5) **Residual.** $r_t = d_i(t) - a_t$ is everything the peers do not back. It is admitted through the residual budget: the accumulated residual R is projected onto the ball $\|R(t)\| \le (B_0 + Bt)\rho_i$.

The spread is handled the same way in log units, with a median over peers, its own agreement and a residual budget $|R_s(t)| \le B_0^s + B^s t$. Defaults are $B_0 = 0.1$, $B = 0.005$ alarm radii per cycle, $B_0^s = 0.1$, $B^s = 0.001$, $B_0^c = 0.75$, $B^c = 0.06$. The spread budget is deliberately tight: on honest traffic the device-specific spread residual stays near zero, while inflating the spread is the cheapest way for an attacker to make an attack fit inside the alarm radius.

Algorithm 1 FOA step on device i in cycle t

$(d, e, g) \leftarrow \textsc{Propose}(X_i(t), \hat{m})$ ▷ signature-free local rule
broadcast $\langle i, t, d, e \rangle$ with HMAC tag ▷ 92 bytes
$\mathcal{F} \leftarrow$ verified, fresh frames from $P(i)$
if $|\mathcal{F}| \ge 2$ **then**
  $m \leftarrow \mathrm{GM}(\{d_j\})$; $\bar{m} \leftarrow (1-\alpha)\bar{m} + \alpha m$
  $\sigma \leftarrow \mathrm{median}_j \left\| d_j - m \right\| / \sqrt{|\mathcal{F}|}$; $\bar{\sigma} \leftarrow (1-\alpha)\bar{\sigma} + \alpha\sigma$
  $w \leftarrow \mathrm{ramp}_{[2,4]}\left(\|\bar{m}\| / (\bar{\sigma}\sqrt{\alpha/(2-\alpha)}) \right)$
  $\bar{g} \leftarrow (1-\alpha)\bar{g} + \alpha g$; $\lambda \leftarrow \mathrm{clip}(\bar{g}/\|\bar{m}\|, 0, 1)$ ▷ robust agreement
  $a \leftarrow \Pi_{(B_0^c + B^c t)\rho}(C + w\lambda\bar{m}) - C$ ▷ common budget
else $a \leftarrow 0$
end if
$r \leftarrow \Pi_{(B_0 + Bt)\rho}(R + d - a) - R$ ▷ residual budget
$C \leftarrow C + a$; $R \leftarrow R + r$; $\mu_i \leftarrow \mu_i + a + r$
spread: same steps on e in log units

D. Authenticated gossip

A proposal travels as one ESP-NOW broadcast of 92 bytes: version, sender, number of learning points, cycle number, sixteen float32 centre shifts, one float32 log-spread change, and an HMAC-SHA256 tag truncated to 128 bits [13] under the sender's provisioned key. Receivers drop frames with an unknown sender, a bad tag or a stale cycle number, which removes forged and replayed proposals. ESP-NOW's own encryption does not cover broadcast, so the tag is ours.

E. Joining the fleet

A new device that calibrates on its own first window can be poisoned before it ever learns, if the attacker is already present. FOA's join rule starts a new device from the geometric median of same-model peers' detector state when such peers exist, and then runs it under the residual budget from zero, so join-time poisoning is bounded by the same $B_0 + Bt$.

IV. Analysis

We state what FOA guarantees, what it does not, and why the remaining trade-off is unavoidable. Throughout, device i has calibrated centre μ^0, spread s^0, threshold τ and alarm radius $\rho = \tau s^0$; t counts cycles since the device joined; $L(t) = B_0 + Bt$ and $L_s(t) = B_0^s + B^s t$. FOA keeps four accumulators: the applied common movement $C(t) \in \mathbb{R}^D$ and $C_s(t) \in \mathbb{R}$, and the applied residual movement $R(t)$ and $R_s(t)$.

A. Containment

Theorem 1 (Residual containment)**.** *For any sequence of batches and peer frames, at every cycle*

$$\mu(t) = \mu^0 + C(t) + R(t), \qquad \log \frac{s(t)}{s^0} = C_s(t) + R_s(t), \quad (2)$$

$$\|R(t)\| \le L(t)\,\rho, \qquad |R_s(t)| \le L_s(t), \quad (3)$$

and each common increment is $a_t = \kappa_t\, w_t \lambda_t \bar{m}_t$ with $\kappa_t \in [0,1]$ (the common budget's shrink factor), where $\bar{m}_t$ and w_t are functions of the verified peer frames only and $\lambda_t \in [0,1]$.

Proof. The device changes μ only by $a_t + r_t$, with a_t added to C and r_t to R; by induction from $\mu(0) = \mu^0$, $C(0) = R(0) = 0$ the decomposition holds, and the same for the log-spread. The residual step sets $R(t)$ to the projection of $R(t-1) + (d_t - a_t)$ onto the ball of radius $L(t)\rho$, so the bound holds after every step; likewise R_s is clipped to $[-L_s(t), L_s(t)]$. Before the common budget, $a_t = w_t \lambda_t \bar{m}_t$; projecting $C + a_t$ onto a ball centred at the origin that contains C returns $C + \kappa_t a_t$ for some $\kappa_t \in [0,1]$. $\bar{m}_t$ is an average of geometric medians of peer proposals and w_t a function of their dispersion, so neither reads the device's own traffic. λ_t is clipped to $[0,1]$. □

The theorem says what the victim's own traffic can and cannot do. Its own traffic enters only through λ_t, which can only *scale down* a peer-determined vector, and through the residual, which is bounded by $L(t)\rho$. It cannot add any direction the peers did not vote for. Because λ_t uses the median projection of the device's core points, an attacker with less than half the batch can neither drive it to zero (to stop the device following the fleet) nor to one (to drag the device along a consensus its own traffic does not show).

Corollary 1 (Minimum time to compromise)**.** *Let $d_{50}(t)$ be the median distance, in calibration spreads, of the target attack traffic from the consensus-backed centre $\mu^0 + C(t)$. If at cycle t fewer than half of that traffic is flagged, then*

$$d_{50}(t) \;\le\; \tau\, e^{C_s(t) + L_s(t)} + \tau L(t). \quad (4)$$

With no common movement ($C = 0$, $C_s = 0$), compromise is impossible before the first t at which $d_{50} \le \tau(e^{B_0^s + B^s t} + B_0 + Bt)$; for a target at M alarm radii and small budgets this is about $t^* \approx (M - 1 - B_0 - B_0^s)/(B + B^s)$ cycles.

Proof. If more than half the attack points x satisfy $\|x - \mu(t)\| \le \tau s(t)$, then for those $\|x - \mu^0 - C(t)\| \le \tau s(t) + \|R(t)\|$, so the median is at most that. Divide by s^0 and apply Theorem 1. The approximation uses $e^y \approx 1 + y$. □

We checked the bound in every cycle of all 3,630 attacked runs of FOA and its budgeted variants and of 120 synthetic runs: it was never violated. The only violations came from the ablation that removes the residual budget, to which the corollary does not apply (Section V-H), which shows the bound is not vacuous.

B. Byzantine peers

Lemma 1 (Minsker [10], Lemma 2.1)**.** *Let $0 < \alpha < 1/2$ and $C_\alpha = (1-\alpha)\sqrt{1/(1-2\alpha)}$. If more than $(1-\alpha)K$ of K points lie within distance r of z, their geometric median lies within $C_\alpha r$ of z.*

Theorem 2 (Bounded Byzantine influence)**.** *Suppose $f < K/2$ of the K peers hold valid keys and send arbitrary frames, and in cycle t the honest peers' proposals lie within r_t of their own geometric median m_t^H. Then $\|m_t - m_t^H\| \le C_{f/K}\, r_t$, so the adversarial part of each common increment is at most $w_t\lambda_t$ times an average of such terms, and the total adversarial common movement is at most* $\min\big(\sum_u C_{f/K} r_u,\ (B_0^c + B^c t)\rho\big)$.

Proof. Apply Lemma 1 with $z = m_t^H$: $K - f > (1 - f/K)K$ points lie within r_t of it. The exponential average is a convex combination, $w_t\lambda_t \le 1$, and the common budget caps $\|C(t)\|$. □

This is a bound on *rate*, not immunity: a minority of compromised peers can still push a consistent bias of order $C_\alpha r$ per cycle until the common budget stops it, and a majority ($f \ge K/2$) can move the consensus anywhere. Keyless attackers have no effect: frames with a bad tag, an unknown sender or a stale cycle are dropped before aggregation.

C. The frontier

Proposition 1 (Device-only drift and poisoning are indistinguishable)**.** *Let any update rule see the device's own traffic and its peers' traffic. Consider world A, in which the device's benign traffic drifts by a vector v per cycle and its peers do not, and world B, in which the device's benign traffic is stationary and an attacker controlling the device's traffic injects traffic distributed exactly as world A's drifted traffic. The observations are identically distributed in both worlds. Hence a rule that keeps false alarms bounded in world A for every v with $\|v\| \le v$ moves the detector by up to v per cycle in world B.*

Proof. The peers' traffic is identical in A and B and the device's traffic has the same law, so any rule produces the same distribution of detector trajectories. Tracking A means following the drift. □

The proposition is the multi-stream counterpart of single-stream impossibility results [22]: peers remove *common* drift from the dilemma but cannot remove *device-only* drift. FOA makes the remaining trade-off a single explicit rate: device-only movement, honest or not, is admitted at rate B (plus B^s in log-spread). Larger B buys faster tracking of a device's own changes and proportionally less time to compromise (Corollary 1); we measure both ends in Section V.

D. No freeze state

Proposition 2 (Freeze immunity)**.** *(i) Traffic that the detector alarms on does not enter FOA's update, so injecting such traffic (decoys) leaves the device's trajectory unchanged. (ii)*

No input sequence stops FOA from learning: the residual admits movement of $B\rho$ per cycle forever.

Proof. (i) The local proposal uses only accepted points (score $\leq \tau$), and the common part depends only on peer frames and the median of accepted core points; alarmed points enter neither. (ii) $L(t)$ grows without bound and nothing in Algorithm 1 sets an update to zero based on the device's alarm rate or any other suspicion statistic. □

Gated defences have exactly the state that Proposition 2 rules out: a condition on a traffic statistic under which they stop learning. An attacker who can make that condition true by sending loud traffic can freeze them, and honest drift then turns the freeze into a lasting flood of false alarms (Section V).

V. Evaluation

We ask five questions. Does FOA follow honest drift (liveness)? Does it stop poisoning, in particular poisoning hidden inside drift (safety)? Does vulnerability diversity matter against a monoculture botnet? Is it immune to the freeze attack and to attacks on its gossip? And what does it cost?

A. Setup

Traffic. We use all nine devices of N-BaIoT [4]: two doorbells (Danmini, Ennio), a thermostat (Ecobee), a baby monitor (Philips), four security cameras (Provision PT-737E and PT-838, SimpleHome XCS7-1002 and -1003) and a webcam (Samsung), each with its real benign capture and its real Mirai and BASHLITE attack traffic (Ennio and Samsung have BASHLITE only). Each row is the 115 Kitsune statistics of one packet [3]. The first 20% of each benign capture calibrates the device's detector; the rest streams in capture order, alternate rows for learning and for measuring false alarms. Each attack type is split into a pool the poisoner draws from and a disjoint held-out set on which detection is measured. The shared 16-dimensional whitened basis keeps 93.2% of the within-device variance.

Fleet. The main fleet has the nine devices, one each; peers are all devices of other vendors. For the monoculture experiment a twelve-device fleet has three instances each of four models; instances differ in calibration subsample, position in the capture and random seed.

Honest drift. We apply drift to the raw Kitsune statistics with physical meaning: packet rate $\times g$ (counts scale by g, inter-packet gaps by $1/g$), b bytes of per-packet overhead in both directions (a tunnel or a new record layer), and network jitter $\times h$. Attack traffic gets the overhead and jitter (same network path) but not the benign rate change. Kitsune's log-scaled statistics barely move under small changes (packet rate $\times 8$ raises a static detector's false alarms only to 22%), so drift strength was fixed by its effect on the static detector alone, never by how any adaptive rule behaves. *Strong* common drift ($g = 64$, $b = 240$, $h = 11.4$, ramped over cycles 40 to 160) takes the static detector to 78% false alarms; *moderate* ($g = 16$, $b = 160$, $h = 5$) to about a third. Device-only drift ($g = 32$, $b = 200$, $h = 7.6$, cycles 60 to 120) models one device's own large change. These are stress tests of a large shared change; Section V-J uses real drift.

Attacker. The poisoner targets the victim's scan traffic (Mirai scan, or BASHLITE scan on the two devices without Mirai), the attack class closest to benign traffic, from cycle 40 on, with share $p \in \{0.1, 0.2, 0.3, 0.4\}$ of the victim's learning batch. Modes: *boundary* (real benign points pulled toward real attack points until just inside the alarm radius), *core* (just inside the core radius), *mimicry* (re-centred copies of benign traffic shifted toward the attack, which looks like drift to the device), *multi-direction*, *consensus-aligned* and *pulsing*.

Baselines. All rules share the same detector and the same local proposal, so differences come from how the proposal is applied. *Static* never adapts. *Naive* applies a mean-and-radius proposal over all accepted traffic. *Single-device* applies our signature-free proposal in full. *Clipping* caps each update's norm (robust-aggregation-style clipping with no memory). *Frozen reserve* learns only from points a frozen copy of the calibrated detector accepts (WARDEN-style screening [22]). *Alarm-rate switch-off* skips updates while the batch alarm rate exceeds five times the clean rate (Kloft and Laskov [2]). *Outlier-ratio gate* skips updates while the alarm rate exceeds three times its own moving average (DriftXpert style [17]). *TOA* is our earlier rule that rations movement toward an *oracle* attack signature. *Same-model consensus* applies the element-wise median of same-model devices' proposals and discards the rest (CIoTA style [6]). The gates are our re-implementations of the gating idea in each paper, not the full published systems.

Metrics. A victim is *compromised* when fewer than half of held-out target attack packets are flagged; time to compromise (TTC) is the first such cycle. False-alarm rate (FPR) is measured on held-out benign traffic each cycle, on the victim and on the rest of the fleet. All results are over 10 seeds (95% confidence intervals where shown); a run is 240 cycles of 200 learning points per device.

Protocol hygiene. Design choices (the robust agreement, the spread budget, the drift strength and the victim rule) were made on development seeds 0 and 1; every number below uses held-out seeds 2 to 11. A device is a victim only if its target attack sits at least 1.3 alarm radii from its own benign traffic under the calibrated static detector (median score over τ). Philips (1.04) and SimpleHome XCS7-1002 (1.16) fail this test; no centroid detector protects an attack that close for long, and they remain in the fleet as peers. (The margins were measured on the held-out attack traffic with the static detector, before any adaptive run; the rule selects victims, it does not tune any method.) Finally, honest drift by itself can carry benign traffic into an attack region: on one camera, even an attacker-free run loses its scan detection under strong drift. Every attacked run is therefore paired with the same run without the attacker, and a victim counts as compromised only when it loses detection that its attacker-free twin keeps.

TABLE I

FALSE ALARMS (%) AFTER HONEST DRIFT, NO ATTACKER. SHARED DRIFT: FLEET MEAN; DEVICE-ONLY: THAT DEVICE. 10 HELD-OUT SEEDS.

Rule	Moderate	Strong	Device-only
Static	32.7	78.3	75.6
Naive	2.6	2.6	2.6
Single device	18.9	24.6	26.0
Clipping	22.3	55.9	61.3
Frozen reserve	24.7	59.3	65.0
Alarm switch-off	21.7	45.0	43.5
Outlier-ratio gate	18.3	24.6	26.7
TOA (oracle)	28.5	38.3	n/a
FOA	6.0	9.0	36.1

B. Liveness: following honest drift

Table I and Figure 2 show false alarms with no attacker. Under strong shared drift a static detector reaches 78%. Every gate that pauses learning on a suspicious statistic pays for it: the alarm-rate switch-off and the frozen reserve end at 45% to 59%. FOA ends at 9.0%, below even single-device adaptation (24.6%), because the fleet consensus is a less noisy estimate of the shared drift than any one device's batch. Only naive learning, which accepts everything and is trivially poisoned, is lower (2.6%). The price shows under device-only drift: FOA admits a device's own change only at rate B, so that device ends at 36% false alarms against 26% for single-device adaptation. This is Proposition 1 made visible, and Section V-G shows how B moves it.

C. Safety: poisoning hidden inside drift

With no drift (Table III), the signature-free local rule already resists most attackers (7% compromised against 36% for naive learning), and FOA keeps that (5%). Drift is what breaks single-device defences: during a strong shared drift the detector must move, and a poisoner moves with it. Table II and Figure 3 show the result over 6,720 runs. Against the classic boundary attacker FOA is compromised in 0% of runs at every share, against 63% for single-device adaptation (up to 84% at share 0.4). Over all three attacker types FOA is compromised in 10.8% of runs, single-device adaptation in 39.2%, the alarm-rate switch-off in 29.9% and the frozen reserve in 27.4%. Only TOA does better (5.5%), because it is handed the true attack signature, and it pays for it with false alarms. FOA is the only rule in the lower-left corner of Figure 3b: few compromises *and* few false alarms.

FOA is not unbreakable. A core-hiding attacker at share 0.4 compromises 34% of victims and a mimicry attacker 29%, almost all on the two Provision cameras, whose scan traffic is the closest to benign among our victims. When FOA is compromised it takes longer: a median of 64 cycles after poisoning starts, against 30 for single-device adaptation. The bound of Corollary 1 was checked in every cycle of every attacked FOA run here and was violated 0 times.

TABLE II

ATTACK HIDDEN IN STRONG SHARED DRIFT: VICTIMS COMPROMISED (%, ATTACK-ATTRIBUTABLE) BY ATTACKER TYPE, AVERAGED OVER SHARES 0.1 TO 0.4, AND FALSE ALARMS AFTER DRIFT (%). 7 VICTIMS, 10 HELD-OUT SEEDS.

Rule	Bound.	Core	Mimic.	All	FPR_v	FPR_{fl}
Naive	83	18	2	35	1.1	2.6
Single device	63	37	18	39	10.3	23.8
Outlier-ratio gate	64	37	18	39	10.5	23.8
Alarm switch-off	36	36	18	30	42.2	43.4
Clipping	14	46	30	30	51.0	54.0
Frozen reserve	1	52	28	27	66.6	57.2
TOA (oracle)	0	10	6	5	41.1	37.0
FOA	0	15	18	11	10.2	7.3

TABLE III

POISONING WITH NO DRIFT: VICTIMS COMPROMISED (%) BY ATTACKER TYPE, AVERAGED OVER SHARES 0.1 TO 0.4.

Rule	Boundary	Core	Mimicry	All
Naive	86	21	1	36
Single device	0	15	7	7
Alarm switch-off	0	15	7	7
Clipping	0	14	1	5
Frozen reserve	0	9	3	4
TOA (oracle)	0	4	0	1
FOA	0	7	7	5

D. Monoculture botnets: why the peers must be diverse

The twelve-device fleet has three instances each of four models. A botnet infects k of the three SimpleHome XCS7-1003 cameras and poisons all of them together during strong drift (Figure 4; without drift no rule was compromised). Same-model consensus, as in CIoTA [6], works while the infected cameras are a minority of their group: with one of three infected it is compromised in 0% of runs. Once two or three of the three are infected, the consensus *is* the botnet, and it is compromised in 100% of runs; FOA restricted to same-model peers fails the same way (100%). FOA with vulnerability-diverse peers is compromised in 0% of runs at every k, as is FOA listening to all peers, because one exploit reaches at most three of eleven peers. This is the head-to-head that separates FOA from same-model consensus. In this fleet, diversity has a cost: a device whose only diverse peers are three other models gets a coarser consensus and higher false alarms than one listening to all peers, so in practice a peer group should be as large as possible subject to no single vulnerability class holding half of it.

E. The freeze attack

An attacker sends loud decoy traffic (real attack packets, pushed out to twice the alarm radius) to one device from cycle 40 to 100, while strong honest drift runs from cycle 40 to 160, and then leaves (Figure 5). Decoys are never learned from by any rule; their only possible effect is on a stop-learning gate. The alarm-rate switch-off closes as soon as decoys arrive, honest drift moves on without it, and when the attacker leaves the alarm rate stays high because the traffic has drifted away:

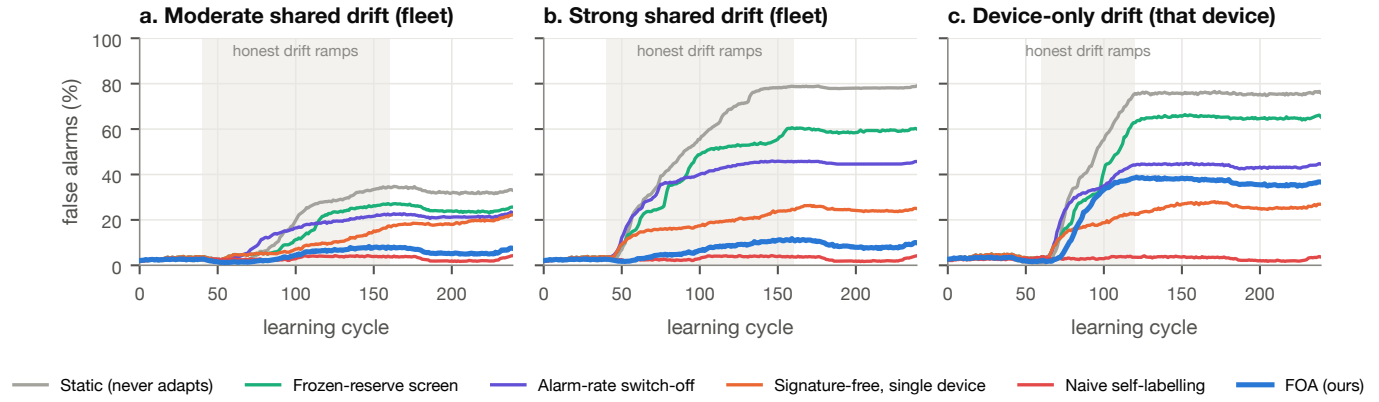


Fig. 2. False alarms with no attacker (10 held-out seeds). Shared drift (a, b) is followed by FOA through the consensus; device-only drift (c) is admitted only at the residual rate B, the cost predicted by Proposition 1.

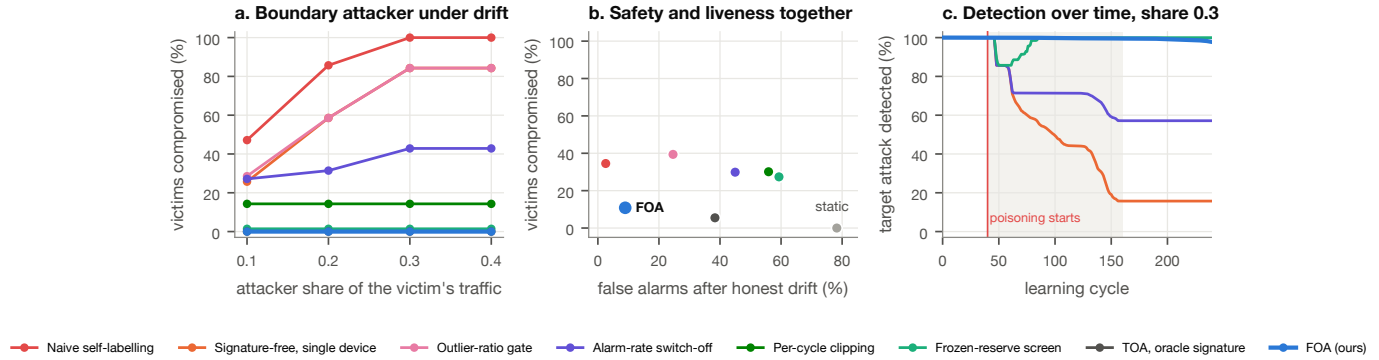


Fig. 3. Poisoning hidden inside strong shared drift (7 victims, 10 held-out seeds, compromise counted against an attacker-free twin). (a) Boundary attacker: FOA is never compromised. The outlier-ratio gate never fires here and coincides with single-device adaptation. (b) Overall compromise against false alarms after drift: FOA alone is low on both. (c) Mean detection of the target attack over time.

TABLE IV

ONE KNOB: RESIDUAL SLOPE B (ALARM RADII PER CYCLE). VICTIMS COMPROMISED UNDER STRONG DRIFT (BOUNDARY ATTACKER, SHARE 0.4, 4 VICTIMS) AND FALSE ALARMS AFTER DEVICE-ONLY DRIFT (%).

B	Compromised	Device-only FPR
0.00125	0	55
0.0025	0	43
0.005	0	31
0.01	50	29
0.02	50	11
0.04	75	16
Single device	75	20
Alarm switch-off	n/a	21
Frozen reserve	n/a	55
Static	n/a	65

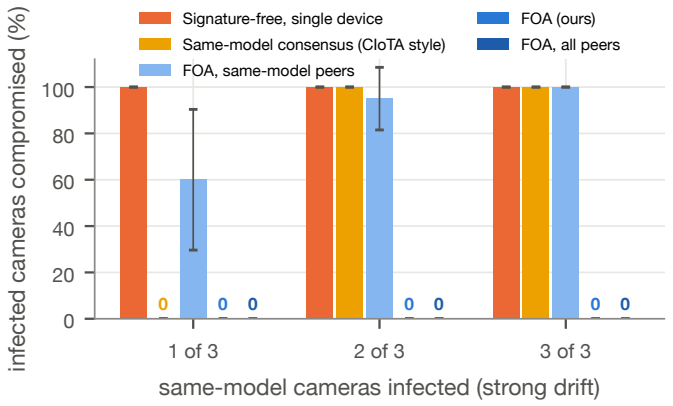


Fig. 4. Monoculture botnet: k of three same-model cameras are infected and poisoned together. Same-model consensus fails as soon as the infected cameras are a majority of their model; FOA with diverse peers does not.

the gate never reopens and the device ends at 100% false alarms, against 26% for the same gate without the attacker, at every decoy share from 5% upward. Not every gate freezes: the outlier-ratio gate compares the alarm rate with its own moving average, which rises with the decoys, so it reopened and ended where it would have without the attacker (26%); the frozen reserve has no gate to trip but suffers from the drift itself. FOA's trajectory is unchanged (11.8% with the attacker, 11.8% without), exactly as Proposition 2 states; our unit tests check that the two trajectories are bit-for-bit identical.

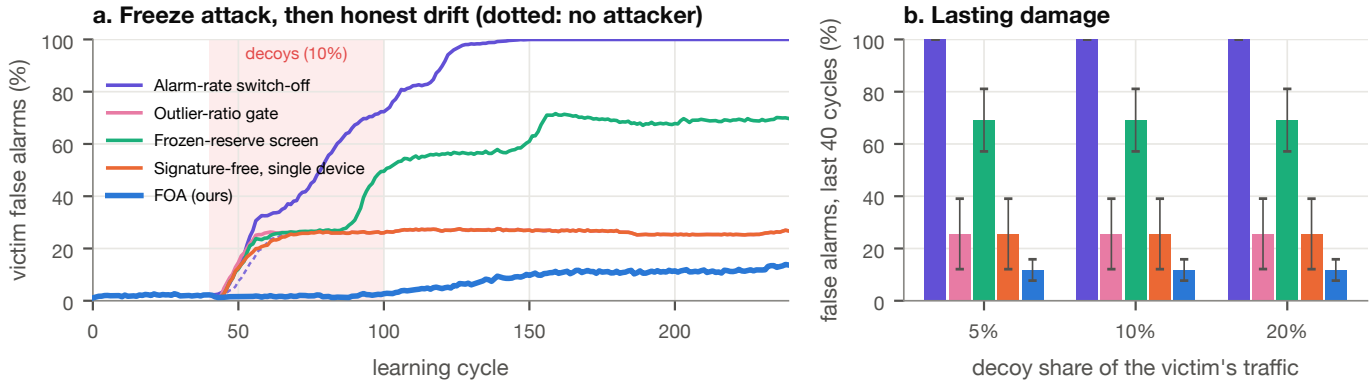


Fig. 5. Freeze attack. (a) False alarms on the targeted device; decoys from cycle 40 to 100 (10% share), honest drift from 40 to 160. Dotted: same rule, no attacker. (b) False alarms over the last 40 cycles by decoy share.

TABLE V
ABLATIONS UNDER STRONG SHARED DRIFT (3 VICTIMS, 10 SEEDS): VICTIMS COMPROMISED BY A BOUNDARY ATTACKER AT SHARE 0.3 (%), FLEET FALSE ALARMS AFTER DRIFT (%), AND FALSE ALARMS ON A DEVICE WITH ITS OWN DRIFT ON TOP OF THE SHARED ONE (%).

Variant	Comp.	FPR_{fl}	FPR_{own}
FOA (default)	0	7.0	90
mean instead of geometric median	0	5.9	74
coordinate-wise median	0	6.9	90
mean-based agreement	0	13.8	90
no agreement ($\lambda = 1$)	0	11.5	90
no significance ($w = 1$)	0	10.1	90
no common budget	0	7.0	90
2× wider spread budget	33	10.0	90
joint centre+spread budget	33	15.2	42
no residual budget	67	23.4	57
discard residual (CIoTA-like)	0	53.6	100
no peers (budget only)	0	43.0	100
2 diverse peers	7	9.7	80
3 diverse peers	3	8.6	80
3 random peers	3	8.1	81
all peers	0	7.2	90

F. Attacks on the gossip

Byzantine peers. We give the attacker full control, keys included, of f of a victim's seven or eight diverse peers, during strong drift, for five victims (Figure 6a). On their own, Byzantine peers compromised no victim at any f up to four (0%): the victim's own traffic does not back what they send, and its agreement λ stays low. Combined with a traffic poisoner on the victim, they erode FOA's protection gradually: 0%, 2%, 56% of victims compromised with 0, 1 and 3 Byzantine peers and 60% with four, against 80% for single-device adaptation under the poisoner alone. The victims that fall are the three closest to their benign traffic (1.5 to 1.9 alarm radii); the two with larger margins were never compromised. Tracing the runs, the extra help travels mostly through the common spread channel: Byzantine frames report spread growth, the poisoned victim's own proposals agree, and a small extra inflation tips a borderline victim. We tried two repairs on the development seeds (letting the device's own median shift choose the common direction, with significance judged from its own noise; and a tighter common spread budget); neither changed this outcome, so we report it as a limit. **Keyless attackers.** Twelve forged frames per cycle plus replays of the previous cycle's frames had no effect (0% compromised; every forged and replayed frame was rejected). With authentication switched off, the same attacker compromised the victim in 100% of runs. **Lossy radio.** Losing up to 70% of frames changed little (fleet false alarms 6.4% with no loss, 6.7% at 50%); at 90% most cycles have fewer than two peers, FOA falls back to its residual budget and false alarms rise.

G. One knob: the residual rate

Table IV sweeps B. With $B \leq 0.005$ alarm radii per cycle no victim was compromised; larger B admits the attacker faster (75% compromised at $B = 0.04$). In exchange, a device's own drift is followed faster: false alarms after device-only drift fall from 55% at the smallest B to 11% at $B = 0.02$. No setting is free, as Proposition 1 says; the bound of Corollary 1 held in every run of the sweep (0 violations). Figure 7 repeats the sweep on a synthetic fleet with known geometry and checks the Byzantine lemma numerically.

H. Ablations and adaptive attackers

Table V removes one piece at a time. Discarding the residual instead of rationing it (the CIoTA choice applied to diverse peers) leaves devices unable to follow anything their peers do not share: fleet false alarms rise to 54%. Removing the peers (budget only) gives 43%. Removing the residual budget gives back the attack (67% compromised). Measuring agreement with a mean instead of a median roughly doubles false alarms (13.8% against 7.0%), because an attacker can then cancel the victim's agreement and stall it. A wider spread budget or a joint centre-and-spread budget lets attackers through: inflating the spread is the cheapest way to make an attack fit. When a device has its own large drift on top of the shared one, almost every variant loses that device (false alarms of 74 to 100%); only the variants with more room (no residual

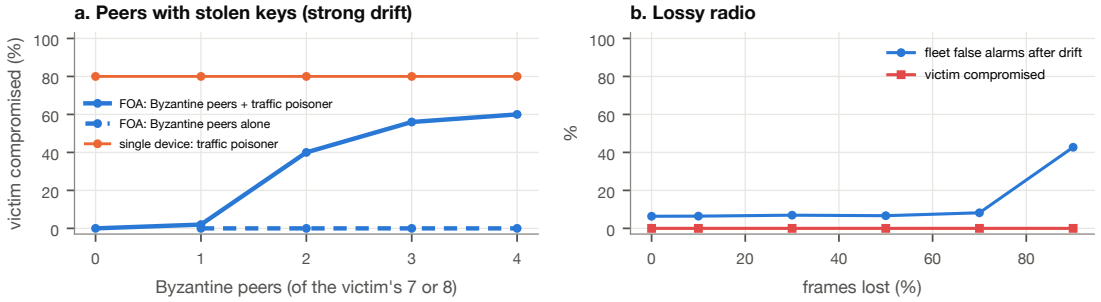


Fig. 6. Attacks on the gossip. (a) Byzantine peers with valid keys, alone and together with a traffic poisoner on the victim (strong drift, 5 victims, 10 held-out seeds). (b) Frame loss.

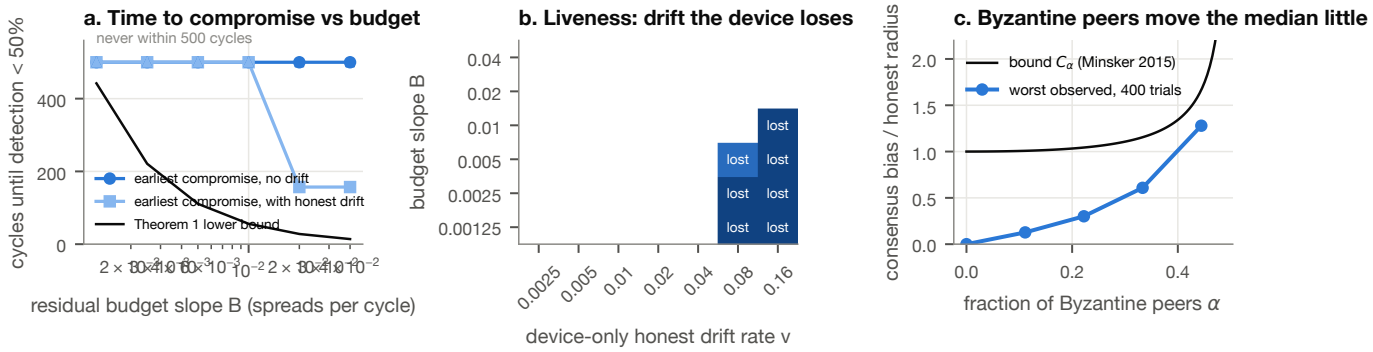


Fig. 7. Theory checks on a synthetic fleet with known geometry. (a) Earliest compromise against the budget slope, with the closed-form lower bound of Corollary 1. (b) Device-only drift rates that a device loses, for each budget slope. (c) Worst observed consensus bias with f Byzantine peers against Lemma 1.

budget, or one joint budget) partly keep up, and both are poisoned. Removing the residual budget is also the only change that ever violated the bound of Corollary 1 (4 runs), as it should. Among adaptive attackers (strong drift, share 0.3, three victims), pulsing compromised no one under either rule; FOA cut the core-hiding attacker from 33% to 10%; and multi-direction, consensus-aligned and mimicry attackers each compromised one of the three victims (33%) under both FOA and single-device adaptation, so FOA did not help against them on that victim.

I. A four-node fleet

The testbed has four ESP32 nodes. Simulating a fleet of four vendors (Danmini, Ecobee, Provision PT-838, SimpleHome XCS7-1003) with strong drift and a boundary attacker at share 0.3: FOA 0% compromised with 11% fleet false alarms; single-device adaptation 75% and 29%; alarm-rate switch-off 50% and 51%; frozen reserve 0% and 77%. Three peers are enough for the consensus to work. Joining devices were never compromised in our join experiment (0%), with or without the warm start.

J. Real drift: 40 weeks of CESNET traffic

The injected drift above is a stress test. To check FOA on drift nobody designed, we use CESNET-TimeSeries24 [5]: traffic statistics of IP addresses in a national research network from October 2023 to July 2024, which span a semester, holidays and the institutions' own changes. The fleet is the twelve most active addresses of one institution (they share its network); a learning cycle is one day of 10-minute aggregates (twelve statistics: flows, packets, bytes, destinations, ports, protocol and direction ratios, duration, TTL), the first 28 days calibrate, and 252 days stream. Nothing about the drift is injected. The attack is modelled, because the dataset has no attack labels: the IP's own traffic turned into a scanner (twenty times the flows, thirty times the destinations, short flows), with a boundary poisoner from day 30. Eleven of the twelve addresses meet the victim rule.

The real drift is milder than our stress test: the static detector stays at 5.9% false alarms, except one address whose behaviour changes for good in the second half and which every rule loses. Every adaptive rule pays something for adapting to daily variation (single device 11.6%, alarm-rate switch-off 13.3%); FOA pays least among the protected rules (9.1%). Under attack, naive learning was compromised in 91% of runs, single-device adaptation and the alarm-rate switch-off in 18% and 18%, the frozen reserve in 0%, and FOA in 0% (Figure 8).

K. Cost on an ESP32

FOA's on-device part is about 500 lines of portable C99 with no heap: the detector, the signature-free proposal with the median agreement projection, ten Weiszfeld iterations, the FOA step, the 92-byte frame and HMAC-SHA256. Table VI summarises what we could verify before the boards arrived.

Equivalence. On 480 recorded cycles from three synthetic fleets (one with 85% frame loss, so 130 cycles had fewer than

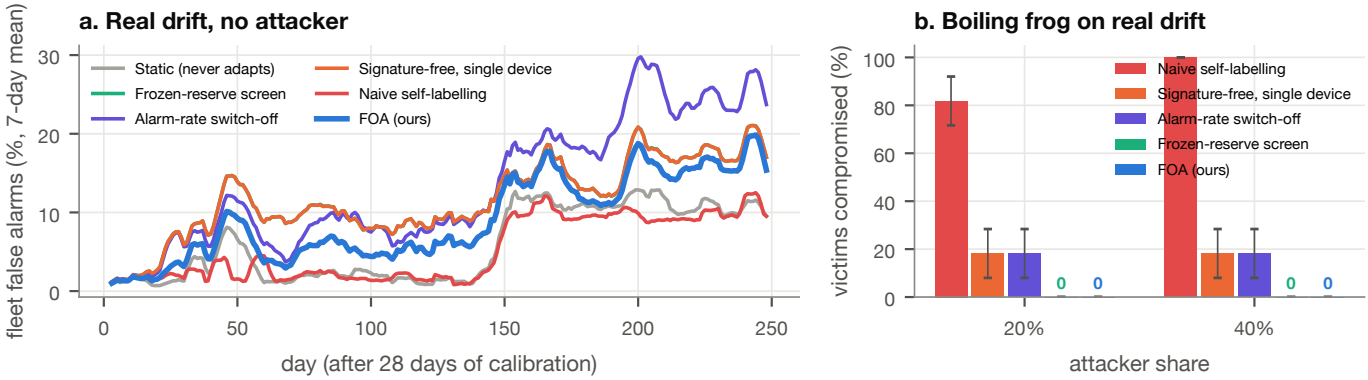


Fig. 8. CESNET-TimeSeries24, twelve addresses of one institution, one learning cycle per day. (a) Fleet false alarms under real drift, no attacker (the frozen reserve never filters here and coincides with single-device adaptation). (b) Victims compromised by a boundary poisoner (11 victims, 5 held-out seeds).

TABLE VI
FOA ON THE ESP32: VERIFIED BEFORE HARDWARE, AND ESTIMATED

C vs Python, 480 cycles	0 mismatches
Tampered frames accepted	0 of 23,104
Added flash / static RAM	22 KB / 20 KB
State per device	452 B
Gossip per device per cycle	one 92 B frame
Compute per cycle (est.)	≈2 ms at 240 MHz

two peers), the C proposal matches the Python reference to a worst relative error of 5.4×10^{-4} (float32 against float64) and the C FOA step to 6×10^{-5}, with zero mismatches. Frames packed in C are byte-identical to Python's; all 23,040 single-bit flips, wrong keys and stale cycle numbers are rejected; HMAC-SHA256 passes RFC 4231 test case 2.

Footprint. Compiled for the ESP32 (Arduino-ESP32 core 3.3.12) the node firmware uses 915,504 bytes of flash and 69,216 bytes of static RAM; the same Wi-Fi, UDP and ESP-NOW code without FOA uses 893,284 and 48,928. FOA therefore costs about 22 KB of flash and 20 KB of static RAM, of which 13 KB is the batch of 200 sixteen-dimensional samples; calibration borrows about 80 KB of heap once and frees it. The detector and FOA state are 452 bytes.

Time and energy (estimates). On the laptop one proposal over 200 points takes 5 μs, one FOA step with 8 peers 0.7 μs and one frame signature 1.2 μs. The step is about $3{\times}10^4$ floating-point operations plus a few hundred square roots; at 240 MHz and a conservative ten cycles per operation this is about 2 ms per learning cycle, against a cycle length of 10 s. One 92-byte ESP-NOW broadcast per cycle is about 1 ms of airtime. These are estimates; the firmware logs the measured time of every stage and the INA219 monitor gives energy per cycle on the testbed.

VI. Related Work

Poisoning self-updating detectors. Rubinstein et al. showed that a network anomaly detector that retrains on its own traffic can be walked toward an attack by slowly increasing chaff, the boiling-frog attack, and defended it with robust PCA on a server [1]. Kloft and Laskov analysed exactly our detector, the online centroid, and showed that the attacker's share of the training traffic bounds how far it can be moved; they also proposed switching learning off when the alarm rate rises [2]. Poisoning of test-time adaptation [12], of on-device learning on microcontroller-class hardware [14], and of rapid-response adaptation pipelines [15] shows the problem is current. A 2026 survey of about seventy on-device learning works for TinyML found no security mechanism among them [16]. FOA targets this gap with a per-device detector that adapts on the device.

Adaptive intrusion detection and its gates. Kitsune made online, per-packet anomaly detection practical but stops learning after training [3]. Later adaptive detectors decide *when* to learn: OWAD detects normality shift and adapts after a test [7]; DriftXpert gates adaptation on an outlier ratio [17]; RTTAD updates only from confidently normal points and stops otherwise [18]; a continual-learning analysis of poisoning proposes rolling back when the score spikes [19]; ACORN-IDS and CND-IDS continue learning from normal-only streams [20], [21]. Each has a condition under which it stops learning. Our freeze experiments show that an attacker who can make such a condition true with loud traffic can turn honest drift into a lasting flood of false alarms. Korycki and Krawczyk discussed attacks that impair adaptation in data streams [11]; we found no prior evaluation of this freeze effect on the gating rules of adaptive intrusion detectors.

Limits of a single stream. Recent theory (all 2026 preprints) shows that one label-free score stream cannot separate honest drift from slow contamination (WARDEN [22]), that regularised continual learners cannot converge under frequent shifted poison [19], and that a single statistic forces a choice between accepting injected changes and rejecting genuine ones (CAPTURE [23]). A further preprint frames self-calibration as an evidence budget with blind sets [24]; we have read only its abstract. FOA does not contradict these results; it changes the observation model by adding peers, and Proposition 1 states what remains impossible.

Collaborative detection and robust aggregation. The closest prior work is CIoTA [6]: devices of the same model

train models collaboratively and accept an update only when the group agrees, discarding the rest. FOA differs in three ways: peers are chosen to be vulnerability-*diverse*, so a monoculture botnet cannot manufacture consensus; the remainder is *rationed* through a budget rather than discarded, which keeps a device able to follow its own changes; and the guarantee is a bound on movement and time to compromise, not agreement. Byzantine-robust aggregation (Krum [8], the geometric median [9], [10]) protects a shared model in federated learning; FOA uses the geometric median only to estimate the common component of per-device updates, and keeps every device's own detector.

VII. Discussion and Limitations

What is real and what is modelled. The benign and attack traffic are real N-BaIoT captures [4], and the C implementation is real and compiled for the ESP32. Three things are ours and synthetic: the concurrency of the fleet (the nine devices were captured separately), the honest drift (physically modelled transforms of the Kitsune statistics, not a recorded network change), and the poisoner (crafted feature vectors, not packets). A public intrusion dataset with documented concept drift did not exist when the 2025 survey of intrusion datasets was written [25], which is why the drift is injected; the testbed's real common-mode drift is the next step.

The frontier cannot be escaped, only placed. Proposition 1 is a limit on every rule, not a weakness of FOA: a device's own drift and an attack on that device look the same to everyone else. FOA's contribution is to move all drift the peers can vouch for out of that dilemma and to make the remaining trade-off one explicit number, B. A deployment that expects frequent device-only changes (firmware updates) can raise B for a bounded window when the update is signed and announced, as licensed-update schemes do; we leave that to future work.

When the fleet is the attacker. If one exploit compromises a majority of a device's peer group, or a gateway rewrites everyone's traffic, consensus carries the attack. Vulnerability-diverse peers make the first harder, not impossible: N-BaIoT itself shows that one credential-guessing botnet (BASHLITE) infected all nine devices [4]. FOA then degrades to its residual budget and common-channel budget, which still bound the rate of movement (Theorem 2), but not to safety.

Limits we measured. Three results are against FOA and we report them as such. Byzantine peers cannot compromise a device on their own, but together with a traffic poisoner they help it: with three of seven peers controlled, about half of the victims fell during strong drift, all of them victims whose attack traffic sits close to their benign traffic. A device with a large drift of its own on top of the shared one is lost by FOA (and by every budgeted variant), while single-device adaptation keeps it. Core-hiding and mimicry attackers at share 0.4 still compromised about a third of victims, mostly the two cameras whose scan traffic is closest to benign.

Spread is the attacker's lever. In the compromised runs we traced, the successful attacks inflated the spread rather than dragging the centre, the radius attack Kloft and Laskov warned about [2]. FOA's spread budget is what holds it, and it is the parameter we would tune first.

Hardware. The boards had not arrived when this paper was written. The firmware is complete and checked against the reference; latency, energy and radio loss on real ESP32s, and the fleet basis fitted on the testbed's own traffic, are reported as host measurements and estimates here and will be replaced by measurements.

VIII. Conclusion

A detector that learns on the device needs a second witness, and an IoT fleet provides one. FOA lets each device take freely whatever drift its vulnerability-diverse peers also see, and rations only what is specific to it. It never stops learning, so it cannot be frozen; it bounds what an attacker on one device can do and how fast; and it costs one 92-byte frame per cycle and about 22 KB of flash on an ESP32. On real IoT traffic with strong shared drift it was the only rule we tested that kept both poisoning and false alarms low. What it cannot do, and what no rule can, is tell a device's own honest change from an attack on that device; FOA turns that limit into a single, explicit rate. The next step is the four-node ESP32 testbed, where the drift, the radio and the energy are real.

Artifacts

Code, data scripts, experiment logs and firmware are in the project repository: the simulator and all experiments (`experiments/foa`), the ESP32 firmware and its equivalence tests (`firmware`), and this paper's number generator (`paper/make_numbers.py`).

Acknowledgment

We thank Dr. Karnish Nasrin Ahmed Tapadar for guidance in CSD457.

References

[1] B. I. P. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S. Lau, S. Rao, N. Taft, and J. D. Tygar, "ANTIDOTE: Understanding and defending against poisoning of anomaly detectors," in *Proc. ACM Internet Measurement Conference (IMC)*, 2009.

[2] M. Kloft and P. Laskov, "Security analysis of online centroid anomaly detection," *Journal of Machine Learning Research*, vol. 13, pp. 3681–3724, 2012.

[3] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. NDSS*, 2018.

[4] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018.

[5] J. Koumar *et al.*, "CESNET-TimeSeries24: Time series dataset for network traffic anomaly detection and forecasting," *Scientific Data*, vol. 12, 2025.

[6] T. Golomb, Y. Mirsky, and Y. Elovici, "CIoTA: Collaborative IoT anomaly detection via blockchain," in *Proc. NDSS Workshop on Decentralized IoT Security and Standards (DISS)*, 2018.

[7] D. Han *et al.*, "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *Proc. NDSS*, 2023.

[8] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017.

[9] H. P. Lopuhaä and P. J. Rousseeuw, "Breakdown points of affine equivariant estimators of multivariate location and covariance matrices," *The Annals of Statistics*, vol. 19, no. 1, pp. 229–248, 1991.
[10] S. Minsker, "Geometric median and robust estimation in Banach spaces," *Bernoulli*, vol. 21, no. 4, pp. 2308–2335, 2015.
[11] Ł. Korycki and B. Krawczyk, "Adversarial concept drift detection under poisoning attacks for robust data stream mining," *Machine Learning*, 2022.
[12] Y. Su *et al.*, "On the adversarial risk of test time adaptation: An investigation into realistic test-time data poisoning," in *Proc. ICLR*, 2025.
[13] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, 1997.
[14] "Data poisoning attack against neural network-based on-device learning anomaly detector by physical attacks on sensors," *Sensors*, vol. 24, no. 19, art. 6416, 2024.
[15] "Rapid Poison: Practical poisoning attacks against the rapid response framework," in *Proc. ICML*, 2026, arXiv:2606.16242.
[16] "What changes after deployment? A survey on on-device learning in TinyML," arXiv:2605.31226, 2026 (preprint).
[17] "Navigating the latent manifold: Proactive concept drift adaptation for resilient NIDS (DriftXpert)," arXiv:2609.08623, 2026 (preprint).
[18] "When normality shifts: Risk-aware test-time adaptation for unsupervised tabular anomaly detection (RTTAD)," arXiv:2605.10242, 2026 (preprint).
[19] "Theory of continual learning against data poisoning attacks," arXiv:2606.29841, 2026 (preprint).
[20] "ACORN-IDS: Adaptive continual novelty detection for intrusion detection systems," arXiv:2602.07291, 2026 (preprint).
[21] "CND-IDS: Continual novelty detection for intrusion detection systems," in *Proc. Design Automation Conference (DAC)*, 2025, arXiv:2502.14094.
[22] "Self-poisoning in adaptive out-of-distribution detection: A sharp-threshold theory and certified label-free calibration (WARDEN)," arXiv:2607.21673, 2026 (preprint).
[23] "CAPTURE: Disentangling preference drift from memory poisoning in personalized LLM agents," arXiv:2609.02265, 2026 (preprint).
[24] "The price of self-calibration: Exact evidence budgets and manufactured blind sets in adaptive monitoring," arXiv:2609.23254, 2026 (preprint; abstract only read).
[25] "Network intrusion datasets: A survey, limitations, and recommendations," *Computers & Security*, vol. 156, 2025, arXiv:2502.06688.
[26] Espressif Systems, "ESP-NOW," ESP-IDF Programming Guide, accessed Oct. 2026.